

**Московский авиационный институт
(национальный исследовательский университет)**

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №3 по курсу «Информационный поиск»

Студент: М.А. Пирязев
Преподаватель: А.А. Кухтичев
Группа: М8О-207Б-22
Дата: 15.12.2025
Оценка:
Подпись:

Москва, 2021

Лабораторная работа №3 “Токенизация, Закон Ципфа, Стемминг”

В данной лабораторной работе реализована токенизация корпуса документов для дальнейшей индексации, выполнен подсчёт частот терминов и построен график распределения «ранг–частота» в логарифмической шкале с наложением закона Ципфа. В качестве дополнительной нормализации добавлен упрощённый стемминг для русского языка.

Также подготовлены автотесты для токенизатора

1. Описание

Целью лабораторной работы является разработка и экспериментальная проверка базового этапа текстовой предобработки, необходимого для построения поискового индекса. В рамках работы требуется реализовать процедуру токенизации — разбиение входных текстов документов на последовательность элементарных единиц (токенов/терминов), которые далее используются в качестве ключей при индексации и поиске. Дополнительно необходимо сформировать набор правил токенизации (включая нормализацию регистра, обработку пунктуации, дефисов и иных пограничных случаев), а также оценить их практическую применимость: указать достоинства, недостатки и примеры ситуаций, в которых выбранные правила приводят к нежелательному результату, и предложить варианты улучшений.

После выполнения токенизации по корпусу документов требуется получить основные статистические характеристики процесса: общее количество выделенных токенов и среднюю длину токена, а также измерить производительность реализации. Для оценки производительности фиксируется время выполнения программы и анализируется зависимость времени работы от объёма входных данных; на основе этих измерений вычисляется скорость токенизации в пересчёте на килобайт входного текста и обсуждается, насколько полученная скорость является приемлемой, а также какие изменения могли бы ускорить обработку (например, оптимизация работы со строками и памятью, уменьшение числа копирований, параллельная обработка документов).

Отдельной задачей является проверка статистических свойств полученного словаря терминов на соответствие закону Ципфа.

2. Исходный код

2.1 Правила токенизации

Токенизация реализована как однократный разбор UTF-8 текста с выделением токенов по следующим правилам:

1) Нормализация символов:

Приведение к нижнему регистру (латиница и кириллица).

Замена «ё» на «е».

2) Что считается токеном

Токеном является последовательность букв кириллицы/латиницы.

Опционально могут включаться цифры, по умолчанию цифры выступают разделителями.

3) Обработка дефиса

Дефис включается в токен только если стоит внутри слова, т.е. слева и справа от него есть «словесные» символы (буква или цифра при `keep_numbers=1`).

4) Смена алфавита внутри «слова»

Если в процессе набора токена происходит смена скрипта (латиница - кириллица), токен принудительно "сбрасывается" и начинается новый. Это снижает количество мусорных гибридных токенов (например, `frameworkи` разбивается на `framework` и `и`).

5) Фильтрация по минимальной длине

Токены короче `min_len` не учитываются (`min_len` настраивается, по умолчанию 1).

2.2 Достоинства и недостатки

Достоинства

Алгоритм линейный по длине текста (один проход), подходит для больших корпусов.

Нормализация регистра и «ё - е» уменьшает размер словаря и улучшает сопоставление термов.

Контроль дефиса снижает шум (не "приклеивает" пунктуацию к словам).

Разделение при смене алфавита уменьшает число артефактов в словаре.

Недостатки:

При `'keep_numbers=0'` числа теряются как отдельные токены (это может ухудшать поиск по версиям/номерам/идентификаторам).

Упрощённые правила не различают сокращения/аббревиатуры

Суффиксный стемминг может давать ложные слияния

Примеры неудачной токенизации и улучшения

Примеры проблемных случаев

C++", "C#" - токен "с". Возможное улучшение: разрешить символы '+' и '#' как часть токена в специальных правилах для языков программирования.

"x86" при `'keep_numbers=0'` → "х" (цифры отбрасываются). Возможное улучшение: включать `'keep_numbers'` для технических корпусов или выделять числа отдельными токенами.

Составные сущности с подчёркиванием (пример: "create_wall_post") распадутся на несколько токенов. Возможное улучшение: добавить правило для '_' как внутрисловного характера (как дефис), если это важно для корпуса.

2.3 Результаты токенизации (статистика)

В результате выполнения программы выводятся:

Количество токенов ('tokens').

Средняя длина токена ('avg_token_len').

Время выполнения ('time_ms').

Скорость токенизации ('KB_per_s').

Количество уникальных терминов ('unique_terms').

Фактические значения на полном корпусе (100к документов)

docs=100000

bytes_in=709401534

tokens=50625784

avg_token_len=6.26639

time_ms=55026

KB_per_s=12590

unique_terms=1563641

Топ-10 терминов по частоте:

1. в : 2263457

2. и : 1605832

3. на : 703922

4. с : 628179

5. по : 318481

6. не : 275517

7. года : 272608

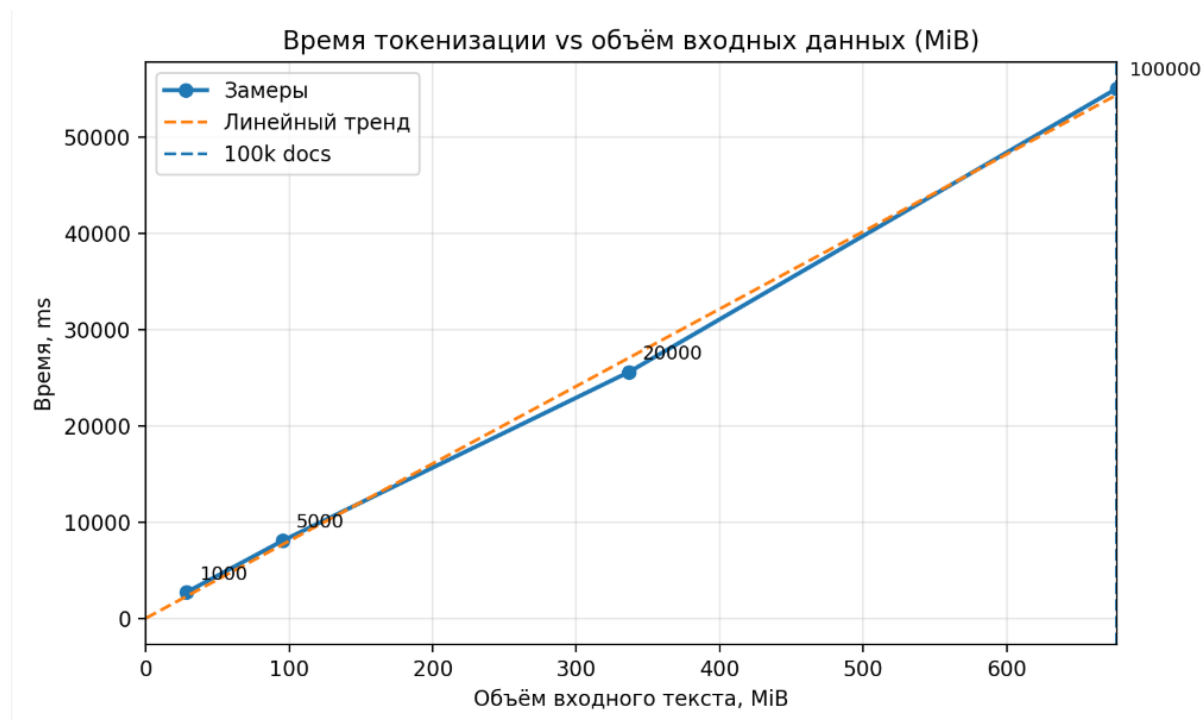
8. из : 259947

9. а : 250791

10. к : 247059

2.5 Зависимость времени от объёма данных

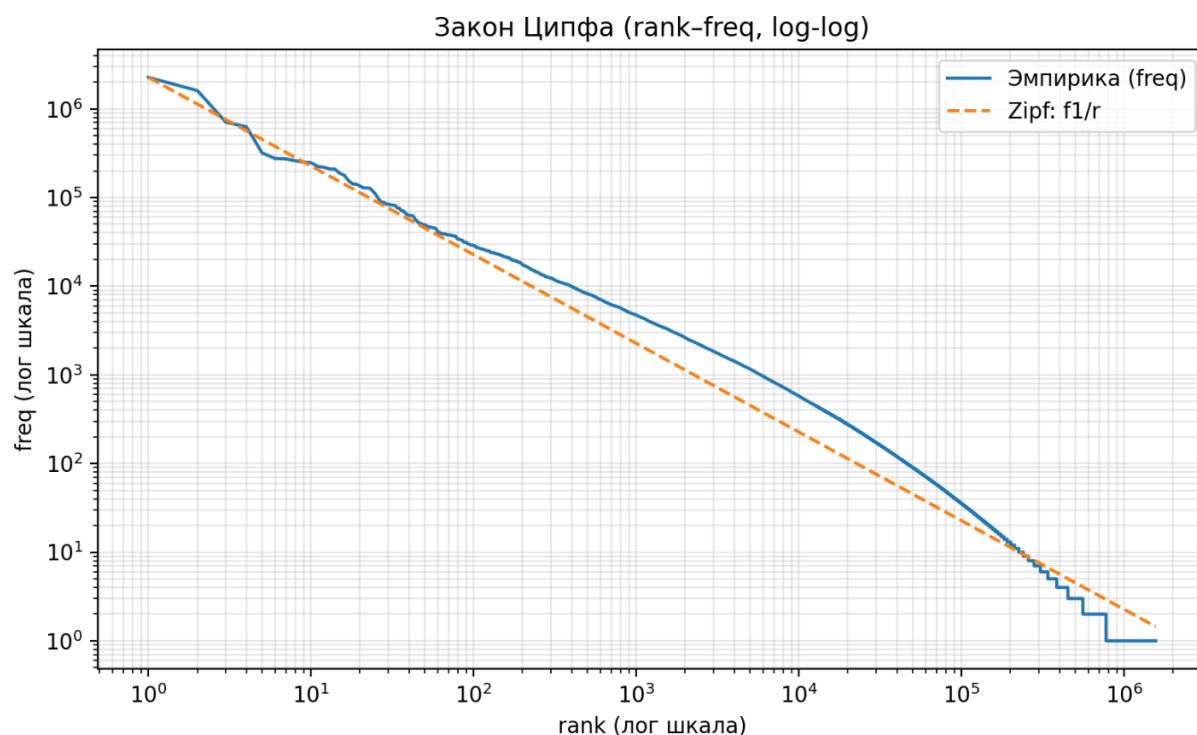
Для демонстрации зависимости времени от объёма входных данных выполняются прогоны с разными ограничениями. Результаты показывают, что время токенизации растёт близко к линейному при увеличении объёма входного текста



Время растёт примерно линейно относительно объёма входных данных. - Средняя скорость токенизации ~12600 KB/s - достаточно быстро для однократного алгоритма. Скорость является оптимальной для текущей реализации.

2.6 Закон Ципфа

После подсчёта частот термины сортируются по убыванию частоты. Для каждого ранга r строится точка $\text{rank}=r$, $\text{freq}=f_r$ в логарифмической шкале, и дополнительно рассчитывается предсказание по простейшей форме закона Ципфа, где f_1 - частота самого частого термина в данном случае $f_1 = 2263457$ для термина "в".



Причины расхождений от идеального закона Ципфа

Служебные слова и артикли ("в", "и", "на", "с") встречаются чаще, чем предсказывает закон Ципфа - это типично для естественного языка.

Смесь доменов (Wikipedia + StackExchange) может привести к нарушению чистого степенного распределения, так как разные источники имеют разные характеристики термов

Хвост редких токенов (опечатки, редкие имена, специфические термины) обычно дает отклонения на больших рангах, так как их частоты не соответствуют простому обратному закону.

Частичная нормализация может привести к появлению близких по смыслу, но технически разных токенов.

2.7 Стемминг

В рамках работы реализован режим нормализации токенов на основе суффиксного стемминга для русского языка. Стемминг применяется после этапа токенизации и базовой нормализации символов (приведение к нижнему регистру, замена «ё» на «е») и включается параметром `--stem=1`. Алгоритм основан на удалении одного из заранее заданных распространённых окончаний русского языка при условии, что после удаления сохраняется минимально допустимая длина основы, что снижает вероятность получения слишком коротких и неинформативных термов.

Используемый подход относится к эвристическим методам и не выполняет полного морфологического анализа: решение о “нормальной форме” принимается исключительно по совпадению с набором суффиксов. К достоинствам данного решения относятся высокая скорость (операции над уже выделенным токеном), простота реализации и заметное снижение вариативности словоформ, что приводит к уменьшению числа уникальных терминов словаря. Кроме того, повышается полнота сопоставления при поиске: запрос, содержащий одну словоформу, чаще находит документы, где встречается другая словоформа того же слова.

Недостатки суффиксного стемминга связаны с потерей точности. Во-первых, возможны ошибки переобобщения, когда разные слова после удаления окончания превращаются в одинаковую основу, что может приводить к росту ложноположительных совпадений. Во-вторых, часть слов может быть “обрезана” неудачно (например, при совпадении с коротким суффиксом), а некоторые формы могут нормализоваться непоследовательно, поскольку алгоритм не учитывает часть речи и контекст. Таким образом, стемминг целесообразно рассматривать как быстрый компромисс между полнотой и точностью: он полезен для повышения recall, но при необходимости высокой точности рекомендуется использовать лемматизацию на основе морфологического словаря/анализатора или комбинированный подход (бонус за точное совпадение формы при сохранении матчинга по основе).

Выводы

В ходе выполнения лабораторной работы реализован модуль токенизации для дальнейшей индексации документов и получены основные метрики качества/производительности (количество токенов, средняя длина токена, время и скорость обработки).

Были сформулированы правила токенизации и рассмотрены типовые недостатки простых правил на смешанном корпусе (Wiki + StackExchange), а также возможные направления улучшений для технических текстов и специфических случаев (обработка аббревиатур, цифр, подчёркиваний).

Был построен частотный словарь и подготовлены данные для графика распределения «ранг–частота» с наложением закона Ципфа, что позволяет проверить адекватность токенизации и статистических свойств корпуса. Расхождения от идеальной степенной функции объясняются смесью источников данных, наличием служебных слов и редких терминов.

Дополнительно реализован упрощённый стемминг, позволяющий уменьшить разнообразие словоформ и в дальнейшем потенциально улучшить полноту поиска, при этом отмечены риски переобобщения, которые следует учитывать при применении стемминга в боевых системах.

Результаты показывают, что текущая реализация токенизации оптимальна по скорости (~12600 KB/s) и может быть использована для индексации больших корпусов. В следующих лабораторных работах эти токены будут использованы для построения обратного индекса и реализации поисковой системы.

Список литературы

- [1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д.А. Ключина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))
- [2] Zipf's law Wikipedia. URL: https://en.wikipedia.org/wiki/Zipf%27s_law (дата обращения: 17.12.2025).